Video Transcript - Part 15

Slide 1

Hey everyone, welcome to part fifteen. In this video we're looking at SageMaker built in algorithms, ready to use machine learning algorithms that come packaged with SageMaker. You don't write the algorithm yourself. You pick the right one, point it at your data, and train. We'll see what's available and how to choose.

Slide 2

Here's the plan. We'll cover what built in algorithms are and how you use them, then an overview of the categories. We'll go through the main supervised, unsupervised, text, forecasting, and image algorithms, and then look at how to choose between them, plus a few practical tips. We'll end with when to use built in algorithms versus your own code.

Slide 3

So what are built in algorithms? They're machine learning algorithms that AWS has already implemented, optimized, and packaged as containers. That means you don't write any training code. They're built to scale, and many support distributed training, so they handle large datasets well. You provide data in S3, choose the algorithm and the hyperparameters, and SageMaker does the rest. They're a great starting point when your problem matches a common pattern.

Slide 4

Using one follows four steps. First, pick the algorithm that fits your problem. Second, prepare your data in a format the algorithm accepts, most commonly CSV or RecordIO protobuf, and put it in S3. Third, set your hyperparameters and choose the instance type and count. Fourth, run the training job, and then deploy the resulting model to an endpoint. Each algorithm's documentation lists the data formats and instance types it supports, so it's worth checking before you start.

Slide 5

There are a lot of built in algorithms, so it helps to group them. Supervised algorithms learn from labeled data, like XGBoost and Linear Learner. Unsupervised algorithms find structure without labels, like K Means and PCA. Text algorithms handle language, like BlazingText and Seq2Seq. DeepAR covers time series forecasting. And there are computer vision algorithms for image classification, object detection, and segmentation. Let's go through the main ones.

Slide 6
Starting with supervised learning. Linear Learner trains linear models for regression and classification. It's simple, fast, and a good baseline. XGBoost is a gradient boosted trees algorithm, and it's one of the most popular choices for tabular data, handling both regression and classification very well. K nearest neighbors predicts based on the closest examples in your data, and works for both classification and regression. And Factorization Machines is designed for sparse data, like recommendations or click prediction, where most of the values are empty.

Slide 7
Now unsupervised algorithms, which don't need labels. K Means groups similar data points into clusters, useful for things like customer segmentation. PCA, principal component analysis, reduces the number of features while keeping most of the information, which is a direct answer to the curse of dimensionality we talked about earlier. Random Cut Forest detects anomalies, like unusual spikes in sensor data or fraudulent activity. And for text, LDA and the Neural Topic Model discover topics in a collection of documents. There's also IP Insights, which flags suspicious behavior around IP addresses.

Slide 8
Next, text, forecasting, and images. BlazingText creates word embeddings and does fast text classification. Seq2Seq turns one sequence into another, which makes it useful for machine translation and summarization. DeepAR is built for forecasting, and it learns from many related time series at once, like sales for many products. And for vision, SageMaker has image classification to label a whole image, object detection to find and box objects, and semantic segmentation to label every pixel.

Slide 9
So how do you choose? If you're predicting a number or a category from a table of data, start with XGBoost, or Linear Learner as a simple baseline. If you want to group similar items without labels, use K Means. If you have too many features, PCA can reduce them. To spot unusual data points, use Random Cut Forest. And to forecast future values of a time series, use DeepAR. Matching the problem type to the algorithm is half the battle.

Slide 10
A few practical tips. Automatic model tuning works with the built in algorithms, so you can search for the best hyperparameters. Check the supported data formats, since many algorithms work best with RecordIO protobuf, and pipe mode can stream that data from S3 to speed up training. Pay attention to instance types too. The image algorithms need GPU instances, while many tabular algorithms run fine on CPU. And remember that built in algorithms still need good data, so everything from earlier videos on cleaning, encoding, and scaling still applies.

Slide 11
When should you use a built-in algorithm? If your problem is a standard one and you want to get going quickly, built in algorithms are ideal. If you need a custom model architecture or special logic, write your own script with a framework like PyTorch or TensorFlow, or bring your own container. And if you want a pretrained model, including foundation models, to start from, JumpStart is the place to look.

Slide 12
So to recap, SageMaker built in algorithms are optimized, ready to use algorithms that save you from writing training code. Supervised ones like XGBoost, unsupervised ones like K Means and PCA, plus text, forecasting, and vision algorithms cover many common problems. Match the problem to the algorithm, check the data format, and tune the hyperparameters. Thanks for watching, and I'll see you in the next one.